Pomme — Cahier des spécifications techniques V1

Nov 4, 2026 · @Ama

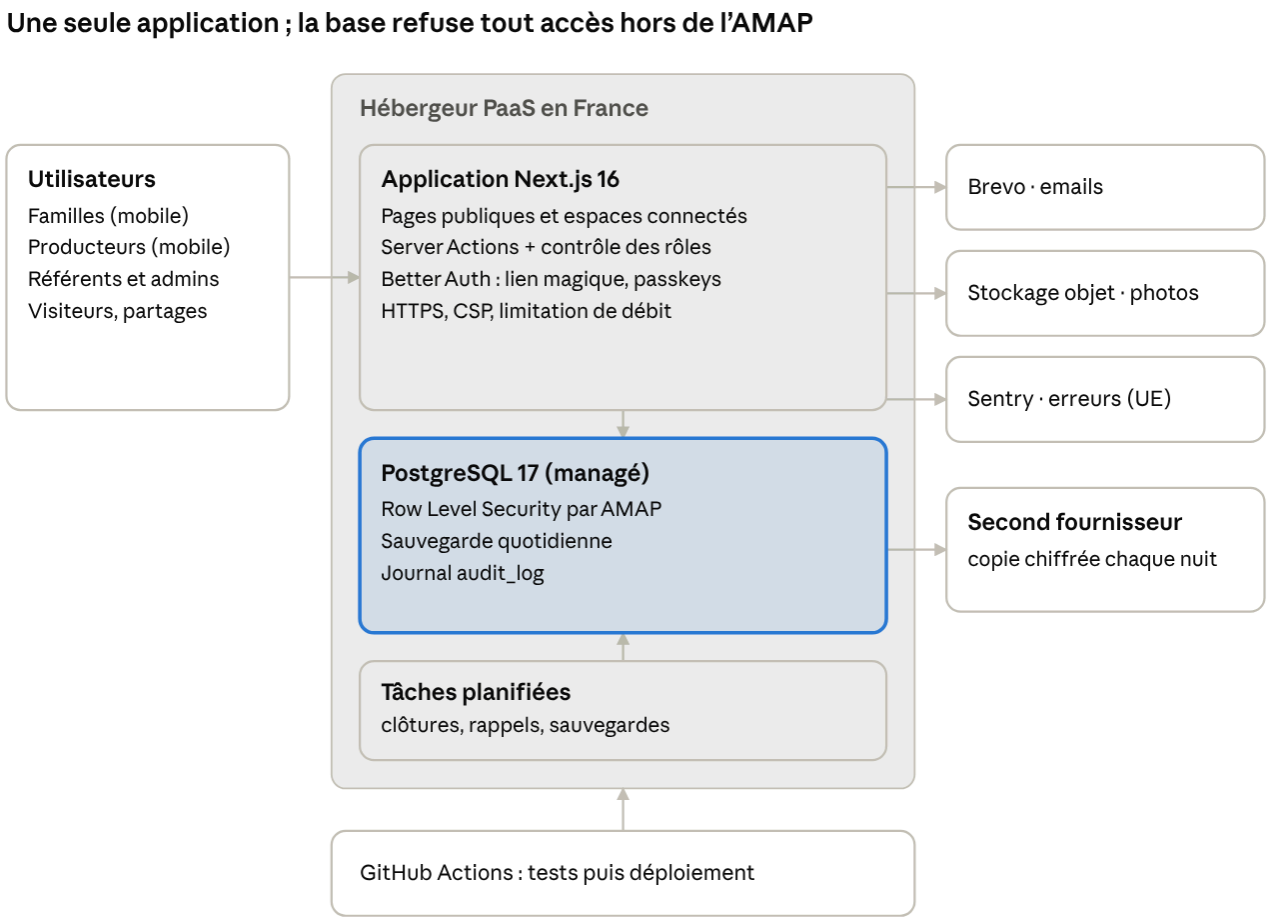
Pomme V1 est une seule application Next.js en TypeScript, avec PostgreSQL, hébergée en France. Quatre impératifs guident les choix : code testé, application sécurisée avec sauvegardes automatiques, interface au plus près de Material Design 3, expérience accessible (clavier, lecteur d'écran). Ce document s'appuie sur le cahier des charges fonctionnel v0 et sur le modèle de données V1 (32 tables).

1. Synthèse des choix

Domaine	Choix recommandé	Raison
Langage	TypeScript strict, partout	Erreurs détectées avant l'exécution ; un seul langage front et back
Application	Next.js 16 (App Router), React 19	Pages publiques rendues côté serveur (référencement, Open Graph) et espaces connectés dans le même projet
Composants	Material UI v9, thème personnalisé « façon M3 »	Composants accessibles et complets ; MUI suit Material Design 2, on le rapproche de M3 par le thème
Couleurs par AMAP	material-color-utilities (Google)	Génère le schéma M3 complet (clair et sombre) à partir des couleurs clés de l'AMAP
Base de données	PostgreSQL 17, Row Level Security par AMAP	Isolation des données garantie par la base, pas seulement par le code
Accès aux données	Drizzle ORM + migrations versionnées	Proche du SQL, déclare les politiques RLS, typage complet
Authentification	Better Auth : lien magique par email, passkeys, 2FA pour les admins	Brique éprouvée et maintenue ; pas de mot de passe à retenir pour les familles
Validation	Zod sur chaque entrée serveur	Aucune donnée non vérifiée n'atteint la base

Domaine	Choix recommandé	Raison
Emails	Brevo (France), envoi transactionnel	Hébergeur européen, offre gratuite suffisante pour 40 foyers
PDF émargement	HTML + CSS d'impression rendu par Playwright côté serveur	Gère la règle « jamais de bloc coupé entre deux pages »
Tests	Vitest (unitaires et intégration), Playwright + axe-core (bout en bout et accessibilité)	Couvre règles métier, base réelle et parcours clavier
Hébergement	Clever Cloud (PaaS français), PostgreSQL managé	Pas de serveur à administrer, sauvegardes incluses, données en France
Sauvegardes	Quotidiennes chez l'hébergeur + copie chiffrée sur Scaleway Object Storage (stockage seul, moins d'1 € par mois), test de restauration mensuel automatique	Une sauvegarde non testée n'est pas une sauvegarde
Intégration continue	GitHub Actions sur le dépôt `blomki/pomme`	Aucun déploiement sans tests au vert
Supervision	Sentry (région UE) + sonde de disponibilité	Erreurs vues avant que les familles ne les signalent

2. Architecture d'ensemble



architecture V1 · une application, une base, quatre services externes

Tout passe par l'application Next.js, seule à parler à la base ; la base applique elle-même l'isolation par AMAP. Rien n'est déployé sans passer par les tests de GitHub Actions.

3. Stack et bibliothèques

Chaque dépendance est figée dans le fichier de verrouillage (`pnpm-lock.yaml`) ; les mises à jour passent par des pull requests automatiques (Renovate) et la CI.

Brique	Version visée	Rôle
Node.js	24 LTS	Exécution du serveur
pnpm	dernière stable	Gestion des dépendances (rapide, strict sur les dépendances fantômes)
Next.js	16.2 (Turbopack par défaut)	Routage, rendu serveur, Server Actions

Brique	Version visée	Rôle
React	19.x	Interface
Material UI	v9 (avril 2026) + `@mui/material-nextjs`	Composants, thème, intégration App Router
@material/material-color-utilities	dernière stable	Palettes tonales et rôles M3 (clair et sombre) depuis les couleurs clés de l'AMAP
Drizzle ORM + drizzle-kit	dernière stable	Requêtes typées, migrations SQL versionnées, politiques RLS
Better Auth	dernière stable, plugins `magicLink`, `passkey`, `twoFactor`	Comptes, sessions, connexion sans mot de passe
Zod	4.x	Validation des formulaires et des Server Actions
React Hook Form	dernière stable	Formulaires (catalogue, commande, pointage)
Tiptap	dernière stable	Éditeur de texte des pages (gras, intertitre, liste, lien, photo), sortie nettoyée
date-fns (locale `fr`)	dernière stable	Dates, fuseau Europe/Paris
Playwright	dernière stable	PDF d'émargement et tests de bout en bout
Vitest + Testing Library	dernière stable	Tests unitaires, composants et intégration
next-intl	dernière stable	Textes externalisés (français en V1, traduction possible)

Pas de bibliothèque d'état global : l'état vit dans la base et dans l'URL ; les Server Components lisent, les Server Actions écrivent.

4. Interface : Material Design 3 et accessibilité

MUI v9 implémente Material Design 2. On obtient l'apparence et le comportement M3 par un thème et quelques composants maison, sans changer de bibliothèque. La référence visuelle reste les écrans Gemini (boutons en pilule, cartes arrondies, palette orange et

vert).

Thème M3 sur MUI

- **Couleurs** : `material-color-utilities` applique le système de couleurs M3. Chaque AMAP fixe ses couleurs clés, comme dans le Material Theme Builder : primaire, secondaire et tertiaire (obligatoires), neutre et neutre variante (facultatives, dérivées de la primaire si elles ne sont pas saisies) ; la couleur d'erreur garde la valeur M3 par défaut. Chaque couleur clé donne une palette tonale (tons 0 à 100), et les rôles M3 (primary, on-primary, primary-container, secondary-container, tertiary, surface, surface-container…) sont pris dans ces palettes aux tons prévus par M3, en clair et en sombre. Ils sont exposés en variables CSS (`cssVariables: true` du thème MUI), donc changer d'AMAP change le thème sans recharger.
- **Mode sombre (V1)** : suit le réglage de l'appareil par défaut, avec un choix manuel clair / sombre / automatique mémorisé par utilisateur ; les deux thèmes passent les tests de contraste et les captures visuelles.
- **Formes** : rayons M3 (8, 12, 16, 28 px), boutons en pilule, cartes à 16 px.
- **Typographie** : échelle M3 (Display, Headline, Title, Body, Label) déclarée dans le thème ; la police des titres est un réglage de chaque AMAP, comme sa couleur (Pattaya pour l'AMAP de Cambremer) ; texte courant en Carlito, commun à toutes les AMAP (police libre OFL, aux mêmes dimensions que Calibri), servie par l'application. Toutes les polices sont des fichiers intégrés au projet et servis par l'application, sans appel à Google Fonts.
- **Élévation** : par teinte de surface (surface-container) plutôt que par ombre, comme en M3.
- **Composants à adapter** : barre de navigation du bas (producteur), boutons « filled tonal », chips de filtre, onglets, états (survol, focus, pressé) avec couche d'état M3 (8 %, 10 %, 10 % d'opacité).
- **Mouvement** : courbes et durées M3 (emphasized, standard ; 200 à 500 ms) pour les transitions d'écran, l'apparition des panneaux et les retours d'action. Tout est désactivé si l'utilisateur a choisi « réduire les animations ».

Expérience dynamique

- **Retour immédiat** : enregistrement optimiste (cocher « Reçu », saisir une quantité) avec retour en arrière et message si le serveur refuse.
- **Totaux en direct** pendant la saisie de commande, sans aller-retour serveur.
- **Snackbars** pour confirmer une action, avec « Annuler » quand c'est possible.
- **Squelettes de chargement** plutôt que des écrans blancs ; navigation instantanée entre onglets.

Accessibilité (objectif RGAA 4.1 / WCAG 2.2 niveau AA)

- Tout est utilisable **au clavier seul** : ordre de tabulation logique, focus toujours visible (anneau M3), lien « Aller au contenu », Échap ferme les dialogues, flèches dans les onglets, grilles et listes.
- Raccourcis pour les tâches répétitives : dans le pointage des chèques, Espace coche « Reçu » et Entrée passe au foyer suivant ; dans la commande, flèches haut et bas ajustent la quantité.
- Composants sémantiques (vrais boutons, liens, champs avec étiquette) ; les icônes seules ont un nom accessible.
- Contrastes vérifiés automatiquement sur chaque thème d'AMAP (4,5:1 pour le texte) ; une couleur d'AMAP trop claire est assombrie par la génération de palette.
- Messages d'erreur reliés aux champs et annoncés aux lecteurs d'écran ; zones mises à jour en direct (totaux, écarts) annoncées poliment.
- Cibles tactiles de 48 px minimum sur mobile ; zoom à 200 % sans perte de contenu.
- Tests : axe-core dans chaque test de bout en bout, plus une recette manuelle au clavier et avec NVDA (Windows) et VoiceOver (iPhone) avant chaque version.

5. Sécurité

L'ancien outil PHP a été piraté plusieurs fois : la sécurité passe avant le confort de développement. Référentiel : OWASP ASVS niveau 2 et Top 10. Principe : chaque protection est doublée (code et base, application et hébergeur).

Menace	Protection
Vol de compte	Lien magique à usage unique valable 15 minutes ; passkeys proposées ; 2FA (TOTP) obligatoire pour les administrateurs ; sessions en cookie `HttpOnly`, `Secure`, `SameSite=Lax`, renouvelées après connexion ; déconnexion de tous les appareils possible
Force brute, spam du formulaire « Nous rejoindre »	Limitation de débit par adresse IP et par email (connexion, demande d'adhésion, envoi d'emails) ; captcha respectueux de la vie privée sur « Nous rejoindre » et la connexion : Friendly Captcha (entreprise allemande, traitement en UE, sans cookie ni pistage, adresses IP hachées, données réservées à la sécurité)

Menace	Protection
Accès aux données d'une autre AMAP ou d'un autre foyer	Contrôle du rôle dans chaque Server Action (une fonction `requireRole` unique) **et** Row Level Security PostgreSQL : chaque requête fixe `app.amap_id` et `app.user_id` dans la transaction, la base refuse le reste ; l'application se connecte avec un rôle sans droit de contourner la RLS
Injection SQL	Requêtes paramétrées (Drizzle), aucune concaténation de SQL
XSS (pages, blog)	React échappe par défaut ; le texte riche de l'éditeur est nettoyé côté serveur (liste blanche de balises) ; Content-Security-Policy stricte avec nonce
CSRF	Server Actions vérifient l'origine ; cookies `SameSite` ; aucune action sensible en GET
Fichiers déposés (photos, logos)	Types et taille contrôlés, image ré-encodée côté serveur (supprime les métadonnées et contenus piégés), stockage objet hors du serveur web
Dépendances vulnérables	Renovate + `pnpm audit` et analyse CodeQL en CI ; build bloqué si faille critique
Fuite de secrets	Secrets uniquement dans les variables de l'hébergeur ; détection de secrets (gitleaks) en CI ; aucune donnée réelle en développement
Fraude ou erreur sur les montants	Journal `audit_log` écrit dans la même transaction que l'action ; montants en centimes ; contraintes CHECK en base
Interception	HTTPS partout, HSTS, en-têtes de sécurité (`X-Content-Type-Options` , `Referrer-Policy` , `Permissions-Policy` , `frame-ancestors 'none'`)

Vérifications : un test automatique par rôle prouve qu'un utilisateur ne lit ni n'écrit les données d'une autre AMAP ou d'un autre foyer ; scan OWASP ZAP sur l'environnement de pré-production avant chaque mise en production ; note A+ visée sur Mozilla Observatory.

Espace gestionnaire (création d'AMAP, story J1) : réservé aux comptes `is_platform_admin`, deuxième vérification obligatoire. L'action de création est la seule à écrire hors du périmètre d'une AMAP : elle passe par une fonction de base dédiée (`SECURITY DEFINER`, droits limités à cette création), jamais par un contournement général de la RLS, et chaque création est journalisée dans `audit_log`. Test dédié : un admin d'AMAP ordinaire ne peut ni ouvrir l'écran ni appeler l'action.

RGPD : données en France, pas de coordonnées bancaires, export et anonymisation d'un foyer sur demande, registre des traitements et mentions légales tenus par l'AMAP.

6. Sauvegardes et reprise après incident

Objectif : perdre au plus 24 heures de données (RPO) et redémarrer en moins de 4 heures (RTO). En pratique, la perte sera de quelques minutes si l'hébergeur propose la restauration à un instant donné.

Quoi	Comment	Conservation
Base PostgreSQL, chez l'hébergeur	Sauvegarde quotidienne automatique du service managé (et restauration à un instant donné si disponible)	Selon l'offre, au moins 7 jours
Base PostgreSQL, copie externe	Chaque nuit, une tâche planifiée lance `pg_dump`, chiffre le fichier (age, clé publique) et l'envoie sur Scaleway Object Storage (Paris), dans un compartiment non effaçable (verrouillage d'objet)	30 sauvegardes quotidiennes + 12 mensuelles
Fichiers déposés (photos, logos)	Copie nocturne du stockage Clever Cloud (Cellar) vers Scaleway	Versions conservées 30 jours
Code et configuration	Dépôt GitHub, infrastructure décrite dans le dépôt	Historique complet

La clé privée de déchiffrement n'est pas sur le serveur : elle est gardée hors ligne par deux personnes (Ama et un administrateur de l'AMAP).

Second fournisseur = stockage seul. Aucune application n'est installée chez Scaleway et il n'y a rien à administrer : c'est un espace de dépôt de fichiers. La tâche qui produit et envoie la copie tourne chez Clever Cloud. Volume attendu pour quelques AMAP : moins de

quelques gigaoctets, soit environ 0,015 € par Go et par mois, donc moins d'un euro par mois. Tarif à vérifier à l'ouverture du compte.

Test de restauration automatique : chaque mois, la CI récupère la dernière copie externe, la restaure dans une base jetable et vérifie quelques comptages (foyers, commandes du mois). Un échec envoie une alerte. Une procédure de reprise écrite (`docs/reprise.md`) décrit les étapes, testée une fois par an à la main.

7. Tests et qualité du code

Aucune modification n'arrive en production sans tests au vert. Les règles d'argent et de planning, où une erreur coûte cher, sont testées en premier et le plus finement.

Niveau	Outil	Ce qui est testé	Exigence
Unitaire	Vitest	Calcul des chèques et des avoirs, statut d'adhésion, « Rien ce mois-ci », génération du planning (équité, nouveau avec un expérimenté, indisponibilités), verrouillages à la réouverture, adresses et redirections des pages	100 % des fonctions métier ; couverture ≥ 90 % du dossier `domain/`
Intégration	Vitest + PostgreSQL réel (Testcontainers)	Server Actions, migrations, contraintes, **isolation RLS par rôle et par AMAP**	Chaque Server Action a au moins un test d'accès autorisé et un d'accès refusé
Composants	Vitest + Testing Library	Formulaires (commande, pointage, catalogue), états d'erreur, comportement clavier	Composants interactifs couverts
Bout en bout	Playwright (Chromium, WebKit mobile)	Parcours critiques : commander, clôturer, pointer les chèques, créer un avoir, générer le planning, publier une page	Un test par parcours des user stories M

Niveau	Outil	Ce qui est testé	Exigence
Accessibilité	axe-core dans Playwright + parcours au clavier seul	Chaque écran des tests de bout en bout	Zéro violation « serious » ou « critical »
Visuel	Captures Playwright comparées	Écrans clés, thème clair et sombre, feuille d'émargement PDF	Revue humaine de tout écart

Qualité du code : TypeScript strict, ESLint (règles React, accessibilité `jsx-a11y`, sécurité), Prettier, revue de chaque pull request avant fusion, commits conventionnels. Données de test : un jeu de données fictif reproductible (2 AMAP, 40 foyers, 6 producteurs) chargé par une commande, jamais de données réelles.

8. Hébergement, environnements, déploiement et supervision

Clever Cloud, PaaS français avec PostgreSQL managé, évite d'administrer un serveur (mises à jour système, pare-feu), ce qui réduit le risque d'intrusion. Budget visé : moins de 20 € par mois en V1, à confirmer sur les grilles tarifaires.

Environnement	Usage	Déploiement
Local	Développement, PostgreSQL en conteneur Docker, emails capturés (Mailpit)	—
Prévisualisation	Une instance par pull request, base jetable avec données fictives	Automatique à l'ouverture de la PR
Pré-production	Recette par Ama et un référent, scan de sécurité	Automatique à chaque fusion sur `main`
Production	[domaine]/hommesdeterre, domaine à définir (emails envoyés depuis une adresse @[domaine])	Manuel (bouton), uniquement depuis une version testée en pré-production

Chaîne CI (GitHub Actions) : vérification des types et du style → tests unitaires et d'intégration → build → tests de bout en bout et accessibilité → analyse de sécurité (CodeQL, audit des dépendances, secrets) → déploiement. Les migrations de base s'appliquent automatiquement au déploiement, toujours de façon rétrocompatible (ajouter avant de supprimer).

Tâches planifiées (cron de l'hébergeur) : ouverture et clôture des périodes de commande, rappels avant clôture, copie externe des sauvegardes, test de restauration mensuel.

Supervision : Sentry (région UE) pour les erreurs serveur et navigateur, sans données personnelles ; sonde de disponibilité toutes les 5 minutes avec alerte par email ; journaux applicatifs conservés 30 jours.

9. Organisation du dépôt et conventions

Les règles métier vivent dans un dossier à part, sans dépendance à Next.js ni à la base, pour être testées simplement.

```
pomme/
  src/
    app/
      (public)/[amap]/...        site public : pages, blog, Nous rejoindre
      (app)/[amap]/famille/...   espace famille (mobile)
      (app)/[amap]/referent/...  espace référent
      (app)/[amap]/admin/...     espace administrateur
      (app)/producteur/...       espace producteur (AMAP courante en
paramètre)
      (app)/plateforme/...       espace gestionnaire : création des AMAP (J1)
    domain/        règles métier pures : chèques, avoirs, planning, adhésions
    server/        Server Actions, requireRole, accès base, emails, PDF
    db/            schéma Drizzle, politiques RLS, migrations, données
fictives
    ui/            thème M3, composants partagés
    i18n/          textes en français
  tests/e2e/       Playwright
  docs/            décisions d'architecture (ADR), procédure de reprise
```

Conventions : code et base en anglais, textes d'interface en français ; une décision technique importante = un court document ADR dans `docs/` ; branche par fonctionnalité, pull request obligatoire, `main` toujours déployable.

10. Décisions prises

Décisions validées par Ama le 4 novembre 2026.

- [x] ~~**Qui code ?** Ama, avec Claude Code. Les ADR et les commentaires expliquent le « pourquoi » en langage simple, pour servir aussi de support d'apprentissage.~~
- [x] ~~**Hébergeur** : Clever Cloud (application Node.js, PostgreSQL managé, stockage Cellar pour les photos). Tarifs à confirmer à l'ouverture du compte.~~

- [x] ~~**Second fournisseur** : Scaleway Object Storage, en stockage seul (aucune application à installer). Il reçoit chaque nuit la sauvegarde chiffrée envoyée depuis Clever Cloud. Coût estimé : moins d'1 € par mois.~~
- [x] ~~**Connexion** : sans mot de passe en V1 (lien magique + passkeys). La story A1 du cahier des charges est à mettre à jour.~~
- [x] ~~**2FA obligatoire pour les administrateurs** : oui (application d'authentification ou passkey).~~
- [x] ~~**Mode sombre** : oui, dès la V1.~~
- [x] ~~**Captcha** : Friendly Captcha, retenu sur critères éthiques : entreprise allemande, traitement dans l'UE, sans cookie ni pistage publicitaire, adresses IP hachées, données utilisées uniquement pour la sécurité ; le navigateur résout un petit calcul au lieu de faire étiqueter des images (pas de travail gratuit pour entraîner une IA, contrairement à reCAPTCHA). Leur politique ne mentionne pas explicitement l'IA : demander une confirmation écrite avant la mise en production. Offre gratuite non commerciale (1 domaine, 1 000 vérifications par mois), suffisante en V1.~~
- [x] ~~**Polices** : titres en Pattaya pour l'AMAP de Cambremer (licence libre SIL OFL 1.1, usage et intégration autorisés). Les fichiers sont placés dans le projet et servis par l'application elle-même, sans rien installer sur le serveur ni appeler Google Fonts. Pattaya n'existe qu'en une graisse : réservée aux titres. Texte courant en Carlito (police libre, mêmes dimensions que Calibri), servie par l'application.~~
- [x] ~~**Couleurs** : système de couleurs Material Design 3 complet. Chaque AMAP saisit ses couleurs clés (primaire, secondaire, tertiaire, et neutres si besoin) ; les palettes tonales et les rôles clair et sombre en découlent.~~
- [ ] **Nom de domaine** : à définir (mapomme.org abandonné avec le nom Amapomme le 2026-11-05). À configurer avant la production : DNS vers Clever Cloud, et SPF, DKIM, DMARC pour les envois Brevo.

Sources

- Next.js 16.2, version stable actuelle
- Introducing Material UI and MUI X v9
- Better Auth vs Lucia vs NextAuth (2026)
- Friendly Captcha, conformité RGPD
- Friendly Captcha, offres gratuites
- Scaleway, tarifs du stockage
- Pattaya, licence SIL OFL 1.1
- Carlito, remplaçant libre de Calibri (CTAN)
- Créer un schéma de couleurs Material Design 3 personnalisé